

IN THIS CHAPTER

- A Simple Class 216
- C++ Objects as Physical Objects 223
- C++ Objects as Data Types 226
- Constructors 227
- Objects as Function Arguments 233
- The Default Copy Constructor 238
- Returning Objects from Functions 240
- A Card-Game Example 243
- Structures and Classes 247
- Classes, Objects, and Memory 247
- Static Class Data 249
- `const` and Classes 252
- What Does It All Mean? 256

And now, the topics you’ve all been waiting for: objects and classes. The preliminaries are out of the way. We’ve learned about structures, which provide a way to group data elements. We’ve examined functions, which organize program actions into named entities. In this chapter we’ll put these ideas together to create classes. We’ll introduce several classes, starting with simple ones and working toward more complicated examples. We’ll focus first on the details of classes and objects. At the end of the chapter we’ll take a wider view, discussing what is to be gained by using the OOP approach.

As you read this chapter you may want to refer back to the concepts introduced in Chapter 1, “The Big Picture.”

A Simple Class

Our first program contains a class and two objects of that class. Although it’s simple, the program demonstrates the syntax and general features of classes in C++. Here’s the listing for the `SMALLOBJ` program:

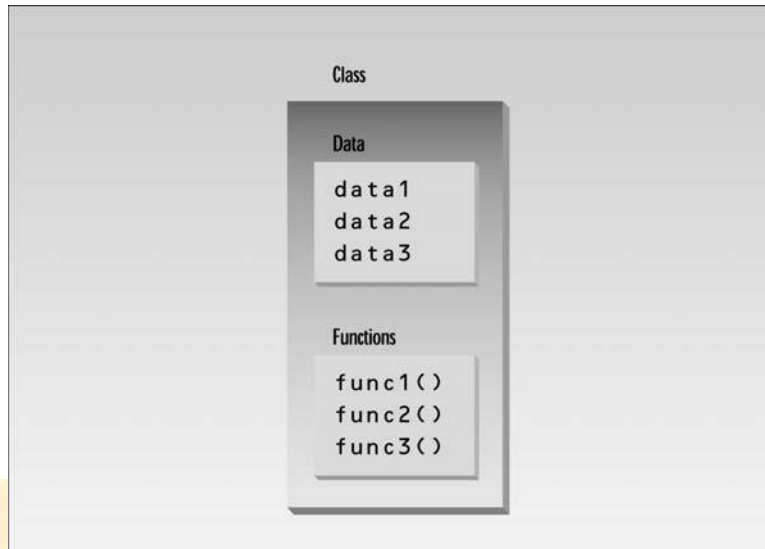
```
// smallobj.cpp
// demonstrates a small, simple object
#include <iostream> using
namespace std;
//////////////////////////////// class smallobj          //define a class
{ private: int somedata;          //class data
public:
    void setdata(int d) //member function to set data
    { somedata = d; } void showdata() //member function to display data
    { cout << "Data is " << somedata << endl; } };
//////////////////////////////// int main() { smallobj s1, s2; //define two objects
of class smallobj

    s1.setdata(1066); //call member function to set data s2.setdata(1776);

    s1.showdata(); //call member function to display data s2.showdata(); return 0;
}
```

The class `smallobj` defined in this program contains one data item and two member functions. The two member functions provide the only access to the data item from outside the class. The first member function sets the data item to a value, and the second displays the value. (This may sound like Greek, but we’ll see what these terms mean as we go along.)

Placing data and functions together into a single entity is a central idea in object-oriented programming. This is shown in Figure 6.1.

**FIGURE 6.1**

Classes contain data and functions.

Classes and Objects

Recall from Chapter 1 that an object has the same relationship to a class that a variable has to a data type. An object is said to be an *instance* of a class, in the same way my 1954 Chevrolet is an instance of a vehicle. In `SMALLOBJ`, the class—whose name is `smallobj`—is defined in the first part of the program. Later, in `main()`, we define two objects—`s1` and `s2`—that are instances of that class.

Each of the two objects is given a value, and each displays its value. Here's the output of the program:

Data is 1066 ←□□□ object s1 displayed this

Data is 1776 ←□□□ object s2 displayed this

We'll begin by looking in detail at the first part of the program—the definition of the class `smallobj`. Later we'll focus on what `main()` does with objects of this class.

Defining the Class

Here's the definition (sometimes called a *specifier*) for the class `smallobj`, copied from the `SMALLOBJ` listing:

```
class smallobj          //define a class
{ private: int somedata;    //class data
public:
    void setdata(int d)    //member function to set data
    { somedata = d; } void showdata()    //member function to display data
    { cout << "\nData is " << somedata; } };
```

The definition starts with the keyword `class`, followed by the class name—`smallobj` in this example. Like a structure, the body of the class is delimited by braces and terminated by a semicolon. (Don't forget the semicolon. Remember, data constructs such as structures and classes end with a semicolon, while control constructs such as functions and loops do not.)

private and public

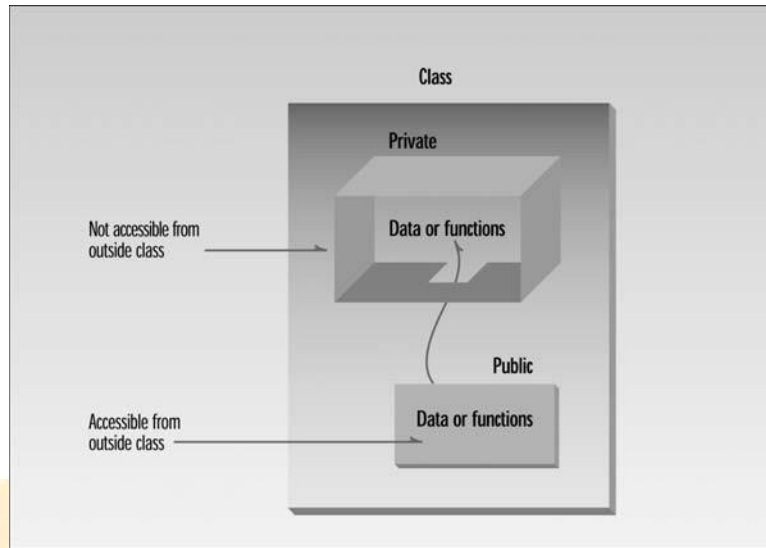
The body of the class contains two unfamiliar keywords: `private` and `public`. What is their purpose?

A key feature of object-oriented programming is *data hiding*. This term does not refer to the activities of particularly paranoid programmers; rather it means that data is concealed within a class so that it cannot be accessed mistakenly by functions outside the class. The primary mechanism for hiding data is to put it in a class and make it private. Private data or functions can only be accessed from within the class. Public data or functions, on the other hand, are accessible from outside the class. This is shown in Figure 6.2.

Hidden from Whom?

Don't confuse data hiding with the security techniques used to protect computer databases. To provide a security measure you might, for example, require a user to supply a password before granting access to a database. The password is meant to keep unauthorized or malevolent users from altering (or often even reading) the data.

Data hiding, on the other hand, means hiding data from parts of the program that don't need to access it. More specifically, one class's data is hidden from other classes. Data hiding is designed to protect well-intentioned programmers from honest mistakes. Programmers who really want to can figure out a way to access private data, but they will find it hard to do so by accident.

**FIGURE 6.2***Private and public.*

Class Data

The `smallobj` class contains one data item: `somedata`, which is of type `int`. The data items within a class are called *data members* (or sometimes *member data*). There can be any number of data members in a class, just as there can be any number of data items in a structure. The data member `somedata` follows the keyword `private`, so it can be accessed from within the class, but not from outside.

Member Functions

Member functions are functions that are included within a class. (In some object-oriented languages, such as Smalltalk, member functions are called *methods*; some writers use this term in C++ as well.) There are two member functions in `smallobj`: `setdata()` and `showdata()`. The function bodies of these functions have been written on the same line as the braces that delimit them.

You could also use the more traditional format for these function definitions:

```
void setdata(int d)
{ somedata = d;
  } and

void showdata()
{ cout << "\nData is " << somedata;
  }
```

However, when member functions are small, it is common to compress their definitions this way to save space.

Because `setdata()` and `showdata()` follow the keyword `public`, they can be accessed from outside the class. We'll see how this is done in a moment. Figure 6.3 shows the syntax of a class definition.

Functions Are Public, Data Is Private

Usually the data within a class is private and the functions are public. This is a result of the way classes are used. The data is hidden so it will be safe from accidental manipulation, while the functions that operate on the data are public so they can be accessed from outside the class. However, there is no rule that says data must be private and functions public; in some circumstances you may find you'll need to use private functions and public data.

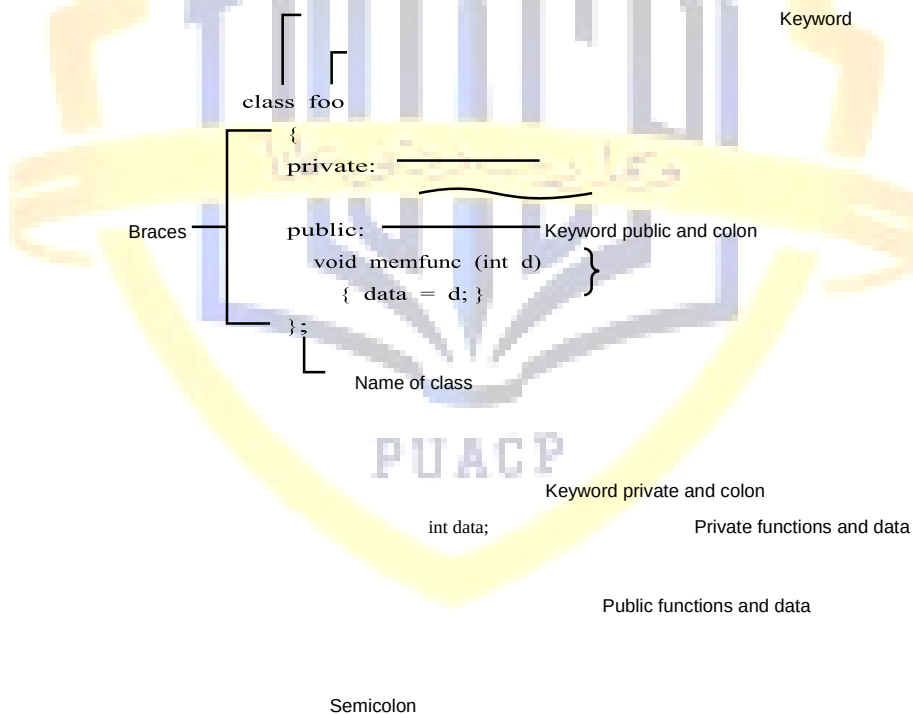


FIGURE 6.3

Syntax of a class definition.

Member Functions Within Class Definition

The member functions in the `smallobj` class perform operations that are quite common in classes: setting and retrieving the data stored in the class. The `setdata()` function accepts a value as a parameter and sets the `somedata` variable to this value. The `showdata()` function displays the value stored in `somedata`.

Note that the member functions `setdata()` and `showdata()` are *definitions* in that the actual code for the function is contained within the class definition. (The functions are not definitions in the sense that memory is set aside for the function code; this doesn't happen until an object of the class is created.) Member functions defined inside a class this way are created as inline functions by default. (Inline functions were discussed in Chapter 5, "Functions.") We'll see later that it is also possible to *declare* a function within a class but to *define* it elsewhere. Functions defined outside the class are not normally inline.

Using the Class

Now that the class is defined, let's see how `main()` makes use of it. We'll see how objects are defined, and, once defined, how their member functions are accessed.

Defining Objects

The first statement in `main()` `smallobj s1,`

```
s2;
```

defines two objects, `s1` and `s2`, of class `smallobj`. Remember that the definition of the class `smallobj` does not create any objects. It only describes how they will look when they are created, just as a structure definition describes how a structure will look but doesn't create any structure variables. It is objects that participate in program operations. Defining an object is similar to defining a variable of any data type: Space is set aside for it in memory.

Defining objects in this way means *creating* them. This is also called *instantiating* them. The term *instantiating* arises because an *instance* of the class is created. An object is an instance (that is, a specific example) of a class. Objects are sometimes called *instance variables*.

Calling Member Functions

The next two statements in `main()` call the member function `setdata()`:

```
s1.setdata(1066); s2.setdata(1776);
```

These statements don't look like normal function calls. Why are the object names `s1` and `s2` connected to the function names with a period? This strange syntax is used to call a member function that is *associated with a specific object*. Because `setdata()` is a member function of the

`smallobj` class, it must always be called in connection with an object of this class. It doesn't make sense to say `setdata(1066)`;

by itself, because a member function is always called to act on a specific object, not on the class in general. Attempting to access the class this way would be like trying to drive the blueprint of a car. Not only does this statement not make sense, but the compiler will issue an error message if you attempt it. Member functions of a class can be accessed only by an object of that class.

To use a member function, the dot operator (the period) connects the object name and the member function. The syntax is similar to the way we refer to structure members, but the parentheses signal that we're executing a member function rather than referring to a data item. (The dot operator is also called the *class member access operator*.) The first call to `setdata()` `s1.setdata(1066)`;

executes the `setdata()` member function of the `s1` object. This function sets the variable `somedata` in object `s1` to the value 1066. The second call `s2.setdata(1776)`;

causes the variable `somedata` in `s2` to be set to 1776. Now we have two objects whose `somedata` variables have different values, as shown in Figure 6.4.

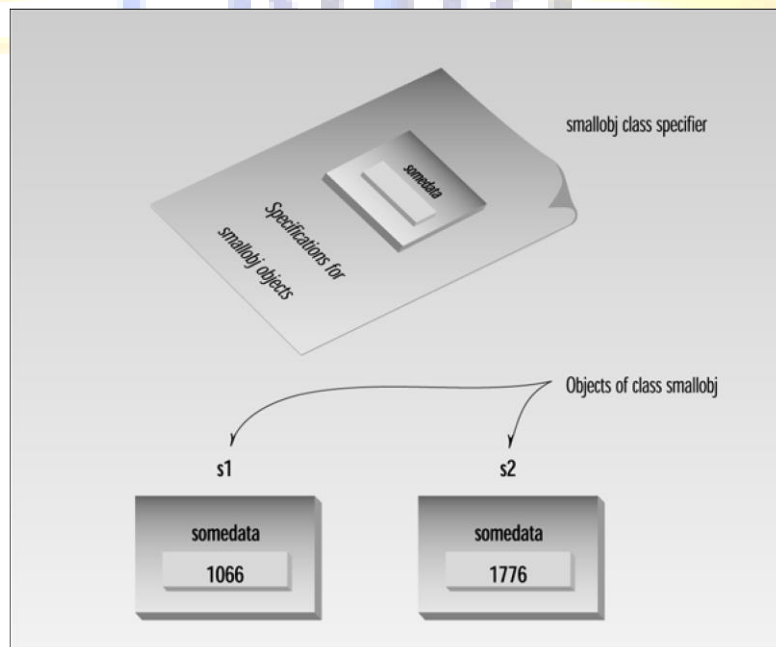


FIGURE 6.4*Two objects of class smallobj.*

Similarly, the following two calls to the `showdata()` function will cause the two objects to display their values:

```
s1.showdata(); s2.showdata();
```

Messages

Some object-oriented languages refer to calls to member functions as *messages*. Thus the call

```
s1.showdata();
```

can be thought of as *sending a message* to `s1` telling it to show its data. The term *message* is not a formal term in C++, but it is a useful idea to keep in mind as we discuss member functions. Talking about messages emphasizes that objects are discrete entities and that we communicate with them by calling their member functions. Referring to the analogy with company organization in Chapter 1, it's like sending a message to the secretary in the sales department asking for a list of products sold in the southwest distribution area.

C++ Objects as Physical Objects

In many programming situations, objects in programs represent physical objects: things that can be felt or seen. These situations provide vivid examples of the correspondence between the program and the real world. We'll look at two such situations: widget parts and graphics circles.

Widget Parts as Objects

The `smallobj` class in the last example had only one data item. Let's look at an example of a somewhat more ambitious class. (These are not the same ambitious classes discussed in political science courses.) We'll create a class based on the structure for the widget parts inventory, last seen in such examples as `PARTS` in Chapter 4, "Structures." Here's the listing for `OBJPART`:

```
// objpart.cpp
// widget part as an object #include
<iostream> using namespace std;
//////////////////////////////////// class part          //define class
{ private:
    int modelnumber; //ID number of widget int partnumber; //ID
    number of widget part float cost; //cost of part
public:
    void setpart(int mn, int pn, float c) //set data
```

```

    {
        modelnumber = mn;
        partnumber = pn; cost = c;
    } void showpart()           //display data
    { cout << "Model " << modelnumber; cout << ",
      part " << partnumber; cout << ", costs $" << cost <<
      endl;
    }

};

//////////////////////////////////// int main() { part part1;           //define object
                                   // of class part
    part1.setpart(6244, 373, 217.55F); //call member function part1.showpart(); //call
    member function return 0;
}

```

This program features the class `part`. Instead of one data item, as `SMALLOBJ` had, this class has three: `modelnumber`, `partnumber`, and `cost`. A single member function, `setpart()`, supplies values to all three data items at once. Another function, `showpart()`, displays the values stored in all three items.

In this example only one object of type `part` is created: `part1`. The member function `setpart()` sets the three data items in this `part` to the values 6244, 373, and 217.55. The member function `showpart()` then displays these values. Here's the output:

```
Model 6244, part 373, costs $217.55
```

This is a somewhat more realistic example than `SMALLOBJ`. If you were designing an inventory program you might actually want to create a class something like `part`. It's an example of a C++ object representing a physical object in the real world—a widget `part`.

Circles as Objects

In our next example we'll examine an object used to represent a circle: the kind of circle displayed on your computer screen. An image isn't quite as tangible an object as a widget `part`, which you can presumably hold in your hand, but you can certainly see such a circle when your program runs.

Our example is an object-oriented version of the `CIRCSTRC` program from Chapter 5. (As in that program, you'll need to add the appropriate Console Graphics Lite files to your project. These files can be downloaded from the publisher's Web site as described in the Introduction. Appendix E, "Console Graphics Lite," describes these files. See also the appendix for your particular compiler.) The program creates three circles with various characteristics and displays them. Here's the listing for `CIRCLES`:

```

// circles.cpp
// circles as graphics objects

```

```

#include "msoftcon.h"    // for graphics functions //////////////////////////////////////
class circle             //graphics circle
{ protected: int xCo, yCo;    //coordinates of center int radius; color fillcolor;
  //color fstyle fillstyle;    //fill pattern public:           //sets circle attributes void
  set(int x, int y, int r, color fc, fstyle fs)
  { xCo = x; yCo = y; radius
    = r; fillcolor = fc; fillstyle
    = fs;
  } void draw()           //draws the circle
  { set_color(fillcolor);    //set color set_fill_style(fillstyle);    //set fill
    draw_circle(xCo, yCo, radius);    //draw solid circle }

};

//////////////////////////////////// int main() { init_graphics();    //initialize graphics
system

    circle c1;             //create circles circle c2; circle c3;

                                //set circle attributes
    c1.set(15, 7, 5, cBLUE, X_FILL); c2.set(41, 12, 7, cRED,
    O_FILL); c3.set(65, 18, 4, cGREEN, MEDIUM_FILL); c1.draw();
    //draw circles c2.draw(); c3.draw(); set_cursor_pos(1, 25);
    //lower left corner return 0;
}

```

The output of this program is the same as that of the `CIRCSTRC` program in Chapter 5, shown in Figure 5.5 in that chapter. You may find it interesting to compare the two programs. In `CIRCLES`, each circle is represented as a C++ object rather than as a combination of a structure variable and an unrelated `circ_draw()` function, as it was in `CIRCSTRC`. Notice in `CIRCLES` how everything connected with a circle—attributes and functions—is brought together in the class definition.

In `CIRCLES`, besides the `draw()` function, the circle class also requires the five-argument `set()` function to set its attributes. We'll see later that it's advantageous to dispense with this function and use a constructor instead.

C++ Objects as Data Types

Here's another kind of entity C++ objects can represent: variables of a user-defined data type. We'll use objects to represent distances measured in the English system, as discussed in Chapter 4. Here's the listing for `ENGLOBJ`:

```

// englobj.cpp
// objects using English measurements
#include <iostream> using
namespace std;

//////////////////////////////////// class Distance           //English Distance class
{ private: int feet; float
  inches;

```

```

public:
    void setdist(int ft, float in) //set Distance to args
    { feet = ft; inches = in; }

    void getdist() //get length from user
    { cout << "\nEnter feet: "; cin >> feet; cout << "Enter
      inches: "; cin >> inches;
    }

    void showdist() //display distance
    { cout << feet << "\'-" << inches << "\'"; }
};

//////////////////////////////////// int main()
{
    Distance dist1, dist2; //define two lengths

    dist1.setdist(11, 6.25); //set dist1 dist2.getdist(); //get dist2 from user

                                //display lengths
    cout << "\ndist1 = "; dist1.showdist(); cout << "\ndist2 = ";
    dist2.showdist(); cout << endl; return 0;
}

```

In this program, the class `Distance` contains two data items, `feet` and `inches`. This is similar to the `Distance` structure seen in examples in Chapter 4, but here the class `Distance` also has three member functions: `setdist()`, which uses arguments to set `feet` and `inches`; `getdist()`, which gets values for `feet` and `inches` from the user at the keyboard; and `showdist()`, which displays the distance in feet-and-inches format.

The value of an object of class `Distance` can thus be set in either of two ways. In `main()`, we define two objects of class `Distance`: `dist1` and `dist2`. The first is given a value using the `setdist()` member function with the arguments `11` and `6.25`, and the second is given a value that is supplied by the user. Here's a sample interaction with the program:

```

Enter feet: 10 Enter inches:
4.75

```

```

dist1 = 11'-6.25" ← provided by arguments dist2 = 10'-4.75"
← input by the user

```

Constructors

The `ENGLOBJ` example shows two ways that member functions can be used to give values to the data items in an object. Sometimes, however, it's convenient if an object can initialize itself when it's first created, without requiring a separate call to a member function. Automatic

initialization is carried out using a special member function called a *constructor*. A constructor is a member function that is executed automatically whenever an object is created. (The term *constructor* is sometimes abbreviated *ctor*, especially in comments in program listings.)

A Counter Example

As an example, we'll create a class of objects that might be useful as a general-purpose programming element. A *counter* is a variable that counts things. Maybe it counts file accesses, or the number of times the user presses the Enter key, or the number of customers entering a bank. Each time such an event takes place, the counter is incremented (1 is added to it). The counter can also be accessed to find the current count.

Let's assume that this counter is important in the program and must be accessed by many different functions. In procedural languages such as C, a counter would probably be implemented as a global variable. However, as we noted in Chapter 1, global variables complicate the program's design and may be modified accidentally. This example, COUNTER, provides a counter variable that can be modified only through its member functions.

```
// counter.cpp
// object represents a counter variable
#include <iostream> using
namespace std;
//////////////////// class Counter { private: unsigned int count;
//count
public:
    Counter() : count(0)           //constructor
    { /*empty body*/ } void inc_count() //increment count
    { count++; } int get_count() //return count
    { return count; }
};
//////////////////// int main()
{
    Counter c1, c2;                //define and initialize

    cout << "\nc1=" << c1.get_count(); //display cout << "\nc2=" <<
    c2.get_count();

    c1.inc_count();                //increment c1 c2.inc_count();          //increment c2
    c2.inc_count();                //increment c2

    cout << "\nc1=" << c1.get_count(); //display again cout << "\nc2=" <<
    c2.get_count();
    cout << endl; return 0;
}
```

The `Counter` class has one data member: `count`, of type `unsigned int` (since the count is always positive). It has three member functions: the constructor `Counter()`, which we'll look at in a moment; `inc_count()`, which adds 1 to `count`; and `get_count()`, which returns the current value of `count`.

Automatic Initialization

When an object of type `Counter` is first created, we want its `count` to be initialized to 0. After all, most counts start at 0. We could provide a `set_count()` function to do this and call it with an argument of 0, or we could provide a `zero_count()` function, which would always set `count` to 0. However, such functions would need to be executed every time we created a `Counter` object.

```
Counter c1;    //every time we do this, c1.zero_count(); //we must do this too
```

This is mistake prone, because the programmer may forget to initialize the object after creating it. It's more reliable and convenient, especially when there are a great many objects of a given class, to cause each object to initialize itself when it's created. In the `Counter` class, the constructor `Counter()` does this. This function is called automatically whenever a new object of type `Counter` is created. Thus in `main()` the statement

```
Counter c1, c2;
```

creates two objects of type `Counter`. As each is created, its constructor, `Counter()`, is executed. This function sets the `count` variable to 0. So the effect of this single statement is to not only create two objects, but also to initialize their `count` variables to 0.

Same Name as the Class

There are some unusual aspects of constructor functions. First, it is no accident that they have exactly the same name (`Counter` in this example) as the class of which they are members. This is one way the compiler knows they are constructors.

Second, no return type is used for constructors. Why not? Since the constructor is called automatically by the system, there's no program for it to return anything to; a return value wouldn't make sense. This is the second way the compiler knows they are constructors.

Initializer List

One of the most common tasks a constructor carries out is initializing data members. In the `Counter` class the constructor must initialize the `count` member to 0. You might think that this would be done in the constructor's function body, like this:

```
count()
{ count = 0; }
```

However, this is not the preferred approach (although it does work). Here's how you should initialize a data member:

```
count() : count(0) { }
```

The initialization takes place following the member function declarator but before the function body. It's preceded by a colon. The value is placed in parentheses following the member data.

If multiple members must be initialized, they're separated by commas. The result is the *initializer list* (sometimes called by other names, such as the *member-initialization list*).

```
someClass() : m1(7), m2(33), m2(4) ← initializer list { }
```

Why not initialize members in the body of the constructor? The reasons are complex, but have to do with the fact that members initialized in the initializer list are given a value before the constructor even starts to execute. This is important in some situations. For example, the initializer list is the only way to initialize `const` member data and references.

Actions more complicated than simple initialization must be carried out in the constructor body, as with ordinary functions.

Counter Output

The `main()` part of this program exercises the `Counter` class by creating two counters, `c1` and `c2`. It causes the counters to display their initial values, which—as arranged by the constructor—are 0. It then increments `c1` once and `c2` twice, and again causes the counters to display themselves (non-criminal behavior in this context). Here's the output:

```
c1=0 c2=0
c1=1 c2=2
```

If this isn't enough proof that the constructor is operating as advertised, we can rewrite the constructor to print a message when it executes.

```
Counter() : count(0)
```

```
{ cout << "I'm the constructor\n"; } }
```

Now the program's output looks like this:

```
I'm the constructor
I'm the constructor
c1=0 c2=0
c1=1 c2=2
```

As you can see, the constructor is executed twice—once for `c1` and once for `c2`—when the statement

```
Counter c1, c2;
```

is executed in `main()`.

Do-It-Yourself Data

Constructors are pretty amazing when you think about it. Whoever writes language compilers (for C or VB or even C++) must execute the equivalent of a constructor when the user defines a variable. If you define an `int`, for example, somewhere there's a constructor allocating four bytes of memory for it. If we can write our own constructors, we can start to take over some of the tasks of a compiler writer. This is one step on the path to creating our own data types, as we'll see later.

A Graphics Example

Let's rewrite our earlier `CIRCLES` example to use a constructor instead of a `set()` function. To handle the initialization of the five attributes of `circles`, this constructor will have five arguments and five items in its initialization list. Here's the listing for `CIRCTOR`:

```
// circtor.cpp
// circles use constructor for initialization
#include "msoftcon.h" // for graphics functions //////////////////////////////////////
class circle          //graphics circle
{ protected: int xCo, yCo; //coordinates of center int radius; color
  fillcolor; //color fstyle fillstyle; //fill pattern public:
  //constructor
  circle(int x, int y, int r, color fc, fstyle fs) :
    xCo(x), yCo(y), radius(r), fillcolor(fc), fillstyle(fs)
  { }

  void draw() //draws the circle
  { set_color(fillcolor); //set color set_fill_style(fillstyle);
    //set fill draw_circle(xCo, yCo, radius); //draw solid circle }
};

//////////////////////////////////// int main() { init_graphics(); //initialize
graphics system
//create circles

circle c1(15, 7, 5, cBLUE, X_FILL); circle c2(41, 12, 7,
cRED, O_FILL); circle c3(65, 18, 4, cGREEN,
MEDIUM_FILL);

c1.draw(); //draw circles c2.draw(); c3.draw();
set_cursor_pos(1, 25); //lower left corner return 0;
}
```

This program is similar to `CIRCLES`, except that `set()` has been replaced by the constructor. Note how this simplifies `main()`. Instead of two separate statements for each object, one to create it and one to set its attributes, now one statement both creates the object and sets its attributes at the same time.

Destructors

We've seen that a special member function—the constructor—is called automatically when an object is first created. You might guess that another function is called automatically when an object is destroyed. This is indeed the case. Such a function is called a *destructor*. A destructor has the same name as the constructor (which is the same as the class name) but is preceded by a tilde:

```
class Foo {
    private:
        int data;
    public:
        Foo() : data(0)    //constructor (same name as class) { }
        ~Foo()            //destructor (same name with tilde) { }
};
```

Like constructors, destructors do not have a return value. They also take no arguments (the assumption being that there's only one way to destroy an object).

The most common use of destructors is to deallocate memory that was allocated for the object by the constructor. We'll investigate these activities in Chapter 10, "Pointers." Until then we won't have much use for destructors.

Objects as Function Arguments

Our next program adds some embellishments to the ENGLOBJ example. It also demonstrates some new aspects of classes: constructor overloading, defining member functions outside the class, and—perhaps most importantly—objects as function arguments. Here's the listing for ENGLCON:

```
// englcon.cpp
// constructors, adds objects using member function
#include <iostream> using
namespace std;
//////////////////////////////////// class Distance            //English Distance class
{ private: int feet; float
  inches;
public:                //constructor (no args)
    Distance() : feet(0), inches(0.0) { }

                                //constructor (two args) Distance(int ft, float in)
    : feet(ft), inches(in)
    { }

    void getdist()            //get length from user
    { cout << "\nEnter feet: "; cin >> feet; cout << "Enter inches:
      "; cin >> inches;
    }
```

```

void showdist()           //display distance
{ cout << feet << "\'-" << inches << "\'"; }

void add_dist( Distance, Distance ); //declaration };

//-----
//add lengths d2 and d3

void Distance::add_dist(Distance d2, Distance d3)
{ inches = d2.inches + d3.inches; //add the inches feet = 0;           //(for
  possible carry) if(inches >= 12.0) //if total exceeds 12.0,
  { //then decrease inches inches -= 12.0; //by 12.0
    and feet++; //increase feet } //by 1 foot += d2.feet
    + d3.feet; //add the feet }

//////////////////////////////// int main()
{
  Distance dist1, dist3; //define two lengths Distance dist2(11, 6.25); //define and
  initialize dist2

  dist1.getdist(); //get dist1 from user dist3.add_dist(dist1, dist2); //dist3 = dist1 + dist2

  //display all lengths
  cout << "\ndist1 = "; dist1.showdist(); cout << "\ndist2 = ";
  dist2.showdist(); cout << "\ndist3 = "; dist3.showdist();
  cout << endl; return 0;
}

```

This program starts with a distance `dist2` set to an initial value and adds to it a distance `dist1`, whose value is supplied by the user, to obtain the sum of the distances. It then displays all three distances:

```

Enter feet: 17 Enter inches:
5.75

```

```

dist1 = 17'-5.75" dist2 = 11'-
6.25" dist3 = 29'-0"

```

Let's see how the new features in this program are implemented.

Overloaded Constructors

It's convenient to be able to give variables of type `Distance` a value when they are first created. That is, we would like to use definitions like

```
Distance width(5, 6.25);
```

which defines an object, `width`, and simultaneously initializes it to a value of 5 for feet and 6.25 for inches.

To do this we write a constructor like this:

```
Distance(int ft, float in) : feet(ft), inches(in) { }
```

This sets the member data `feet` and `inches` to whatever values are passed as arguments to the constructor. So far so good.

However, we also want to define variables of type `Distance` without initializing them, as we did in `ENGLOBJ`.

```
Distance dist1, dist2;
```

In that program there was no constructor, but our definitions worked just fine. How could they work without a constructor? Because an implicit no-argument constructor is built into the program automatically by the compiler, and it's this constructor that created the objects, even though we didn't define it in the class. This no-argument constructor is called the *default constructor*. If it weren't created automatically by the compiler, you wouldn't be able to create objects of a class for which no constructor was defined.

Often we want to initialize data members in the default (no-argument) constructor as well. If we let the default constructor do it, we don't really know what values the data members may be given. If we care what values they may be given, we need to explicitly define the constructor. In `ENGLECON` we show how this looks:

```
Distance() : feet(0), inches(0.0) //default constructor  
{ } //no function body, doesn't do anything
```

The data members are initialized to constant values, in this case the integer value 0 and the float value 0.0, for `feet` and `inches` respectively. Now we can use objects initialized with the no-argument constructor and be confident that they represent no distance (0 feet plus 0.0 inches) rather than some arbitrary value.

Since there are now two explicit constructors with the same name, `Distance()`, we say the constructor is *overloaded*. Which of the two constructors is executed when an object is created depends on how many arguments are used in the definition:

```
Distance length; // calls first constructor  
Distance width(11, 6.0); // calls second constructor
```

Member Functions Defined Outside the Class

So far we've seen member functions that were defined inside the class definition. This need not always be the case. `ENGLCON` shows a member function, `add_dist()`, that is not defined within the `Distance` class definition. It is only *declared* inside the class, with the statement `void add_dist( Distance, Distance );`

This tells the compiler that this function is a member of the class but that it will be defined outside the class declaration, someplace else in the listing.

In ENGLCON the `add_dist()` function is defined following the class definition. It is adapted from the ENGLSTRC program in Chapter 4:

```

//add lengths d2 and d3
void Distance::add_dist(Distance d2, Distance d3)
{ inches = d2.inches + d3.inches; //add the inches feet = 0;           //(for
  possible carry) if(inches >= 12.0) //if total exceeds 12.0,
    { //then decrease inches inches -= 12.0; //by 12.0
    and feet++; //increase feet } //by 1 feet += d2.feet +
    d3.feet; //add the feet }

```

The declarator in this definition contains some unfamiliar syntax. The function name, `add_dist()`, is preceded by the class name, `Distance`, and a new symbol—the double colon (`::`). This symbol is called the *scope resolution operator*. It is a way of specifying what class something is associated with. In this situation, `Distance::add_dist()` means “the `add_dist()` member function of the `Distance` class.” Figure 6.5 shows its usage.

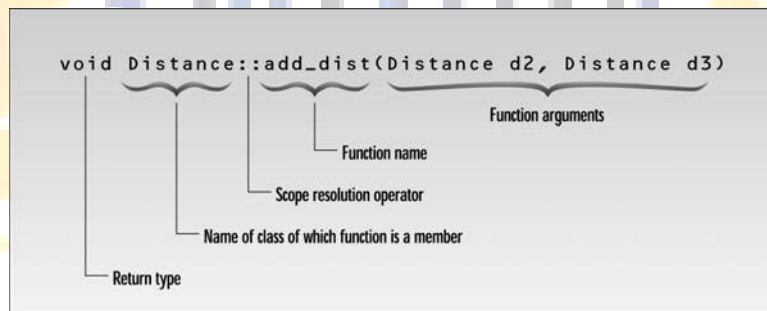


FIGURE 6.5

The scope resolution operator.

Objects as Arguments

Now we can see how ENGLCON works. The distances `dist1` and `dist3` are created using the default constructor (the one that takes no arguments). The distance `dist2` is created with the constructor that takes two arguments, and is initialized to the values passed in these arguments. A value is obtained for `dist1` by calling the member function `getdist()`, which obtains values from the user. Now we want to add `dist1` and `dist2` to obtain `dist3`. The function call in `main()` `dist3.add_dist(dist1, dist2);`

does this. The two distances to be added, `dist1` and `dist2`, are supplied as arguments to `add_dist()`. The syntax for arguments that are objects is the same as that for arguments that are simple data types such as `int`: The object name is supplied as the argument. Since `add_dist()` is a member

function of the `Distance` class, it can access the private data in any object of class `Distance` supplied to it as an argument, using names like `dist1.inches` and `dist2.feet`.

Close examination of `add_dist()` emphasizes some important truths about member functions. A member function is always given access to the object for which it was called: the object connected to it with the dot operator. But it may be able to access other objects. In the following statement in `ENGLCON`, what objects can `add_dist()` access? `dist3.add_dist(dist1, dist2);`

Besides `dist3`, the object for which it was called, it can also access `dist1` and `dist2`, because they are supplied as arguments. You might think of `dist3` as a sort of phantom argument; the member function always has access to it, even though it is not supplied as an argument. That's what this statement means: "Execute the `add_dist()` member function of `dist3`." When the variables `feet` and `inches` are referred to within this function, they refer to `dist3.feet` and `dist3.inches`.

Notice that the result is not returned by the function. The return type of `add_dist()` is `void`. The result is stored automatically in the `dist3` object. Figure 6.6 shows the two distances `dist1` and `dist2` being added together, with the result stored in `dist3`.

To summarize, every call to a member function is associated with a particular object (unless it's a static function; we'll get to that later). Using the member names alone (`feet` and `inches`), the function has direct access to all the members, whether private or public, of that object. It also has indirect access, using the object name and the member name, connected with the dot operator (`dist1.inches` or `dist2.feet`) to other objects of the same class that are passed as arguments.

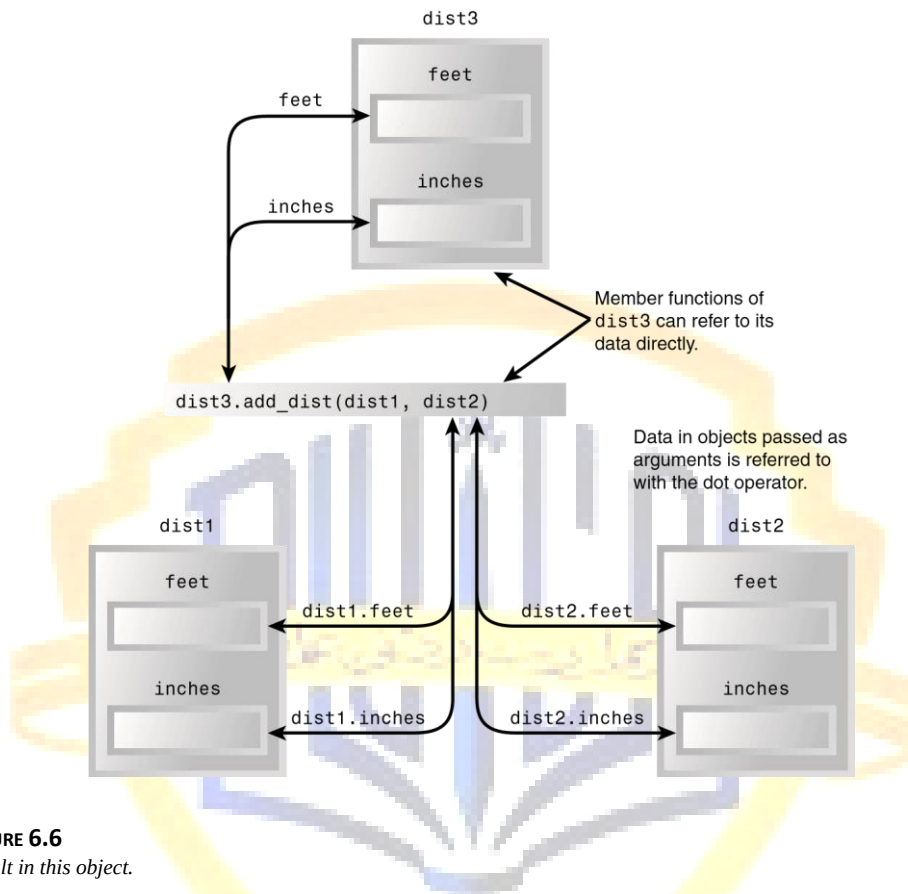


FIGURE 6.6
Result in this object.

The Default Copy Constructor

We've seen two ways to initialize objects. A no-argument constructor can initialize data members to constant values, and a multi-argument constructor can initialize data members to values passed as arguments. Let's mention another way to initialize an object: you can initialize it with *another object of the same type*. Surprisingly, you don't need to create a special constructor for this; one is already built into all classes. It's called the *default copy constructor*. It's a one-argument constructor whose argument is an object of the same class as the constructor. The `ECOPYCON` program shows how this constructor is used.

```
// ecopycon.cpp
// initialize objects using default copy constructor
#include <iostream> using
namespace std;
```

```

////////////////////////////////////
class Distance          //English Distance class
{ private: int feet; float
  inches; public:

                                //constructor (no args) Distance() :
    feet(0), inches(0.0) { }
    //Note: no one-arg constructor

                                //constructor (two args) Distance(int ft, float in) :
    feet(ft), inches(in)
    { }

    void getdist()          //get length from user
    { cout << "\nEnter feet: "; cin >> feet; cout << "Enter inches:
      "; cin >> inches;
    } void showdist()        //display distance
    { cout << feet << "\'-" << inches << "\'"; } };
//////////////////////////////////// int main()
{
    Distance dist1(11, 6.25); //two-arg constructor
    Distance dist2(dist1);    //one-arg constructor
    Distance dist3 = dist1;   //also one-arg constructor

                                //display all lengths
    cout << "\ndist1 = "; dist1.showdist(); cout << "\ndist2 = ";
    dist2.showdist(); cout << "\ndist3 = "; dist3.showdist(); cout
    << endl; return 0;
}

```

We initialize `dist1` to the value of 11'-6.25" using the two-argument constructor. Then we define two more objects of type `Distance`, `dist2` and `dist3`, initializing both to the value of `dist1`. You might think this would require us to define a one-argument constructor, but initializing an object with another object of the same type is a special case. These definitions both use the default copy constructor. The object `dist2` is initialized in the statement

```
Distance dist2(dist1);
```

This causes the default copy constructor for the `Distance` class to perform a member-by-member copy of `dist1` into `dist2`. Surprisingly, a different format has exactly the same effect, causing `dist1` to be copied member-by-member into `dist3`:

```
Distance dist3 = dist1;
```

Although this looks like an assignment statement, it is not. Both formats invoke the default copy constructor, and can be used interchangeably. Here's the output from the program:

```
dist1 = 11'-6.25" dist2 = 11'-
6.25" dist3 = 11'-6.25"
```

This shows that the `dist2` and `dist3` objects have been initialized to the same value as `dist1`. In Chapter 11, “Virtual Functions,” we discuss how to create your own custom copy constructor by overloading the default.

Returning Objects from Functions

In the `ENGLCON` example, we saw objects being passed as arguments to functions. Now we’ll see an example of a function that returns an object. We’ll modify the `ENGLCON` program to produce `ENGLRET`:

```
// englret.cpp
// function returns value of type Distance
#include <iostream> using
namespace std;
//////////////////////////////// class Distance //English Distance class
{ private: int feet; float
  inches;
public:
    Distance() : feet(0), inches(0.0) //constructor (no args)
    { } //constructor (two args)
    Distance(int ft, float in) : feet(ft), inches(in)
    { }

    void getdist() //get length from user
    { cout << "\nEnter feet: "; cin >> feet; cout << "Enter inches:
      "; cin >> inches;
    } void showdist() //display distance
    { cout << feet << "\'-" << inches << "\'"; }
    Distance add_dist(Distance); //add
};

//-----
//add this distance to d2, return the sum
Distance Distance::add_dist(Distance d2)
{
    Distance temp; //temporary variable temp.inches = inches + d2.inches; //add the
    inches if(temp.inches >= 12.0) //if total exceeds 12.0,
    { //then decrease inches temp.inches -= 12.0; //by 12.0
      and temp.feet = 1; //increase feet } //by 1 temp.feet +=
      feet + d2.feet; //add the feet return temp;
    }
}

//////////////////////////////// int main()
{
    Distance dist1, dist3; //define two lengths Distance dist2(11, 6.25); //define,
    initialize dist2
```

```

dist1.getdist();           //get dist1 from user dist3 = dist1.add_dist(dist2); //dist3 =
dist1 + dist2

                                //display all lengths
cout << "\ndist1 = "; dist1.showdist(); cout << "\ndist2 = ";
dist2.showdist(); cout << "\ndist3 = "; dist3.showdist(); cout
<< endl; return 0;
}

```

The ENGLRET program is very similar to ENGLCON, but the differences reveal important aspects of how functions work with objects.

Arguments and Objects

In ENGLCON, two distances were passed to `add_dist()` as arguments, and the result was stored in the object of which `add_dist()` was a member, namely `dist3`. In ENGLRET, one distance, `dist2`, is passed to `add_dist()` as an argument. It is added to the object, `dist1`, of which `add_dist()` is a member, and the result is returned from the function. In `main()`, the result is assigned to `dist3` in the statement `dist3 = dist1.add_dist(dist2);`

The effect is the same as the corresponding statement in ENGLCON, but it is somewhat more natural looking, since the assignment operator, `=`, is used in a natural way. In Chapter 8, “Operator Overloading,” we’ll see how to use the arithmetic `+` operator to achieve the even more natural expression `dist3 = dist1 + dist2;`

Here’s the `add_dist()` function from ENGLRET:

```

//add this distance to d2, return the sum
Distance Distance::add_dist(Distance d2)
{
    Distance temp;           //temporary variable temp.inches = inches + d2.inches; //add the
    inches if(temp.inches >= 12.0) //if total exceeds 12.0,
    {
        //then decrease inches temp.inches -= 12.0; //by 12.0
        and temp.feet = 1; //increase feet } //by 1 temp.feet +=
        feet + d2.feet; //add the feet return temp;
    }
}

```

Compare this with the same function in ENGLCON. As you can see, there are some subtle differences. In the ENGLRET version, a temporary object of class `Distance` is created. This object holds the sum until it can be returned to the calling program. The sum is calculated by adding two distances. The first is the object of which `add_dist()` is a member, `dist1`. Its member data is accessed in the function as `feet` and `inches`. The second is the object passed as an argument, `dist2`. Its member data is accessed as `d2.feet` and `d2.inches`. The result is stored in `temp` and accessed as `temp.feet` and `temp.inches`. The `temp` object is then returned by the function using the statement `return temp;`

and the statement in `main()` assigns it to `dist3`. Notice that `dist1` is not modified; it simply supplies data to `add_dist()`. Figure 6.7 shows how this looks.

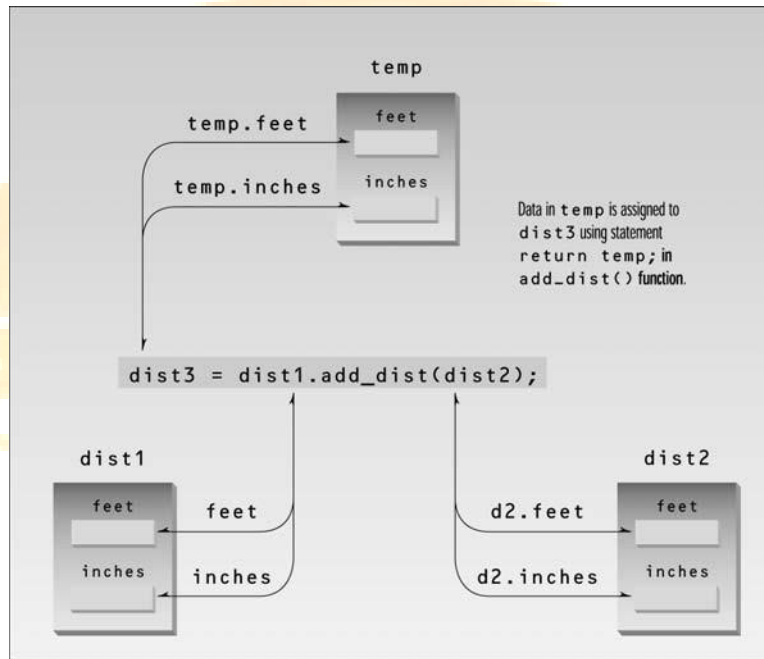


FIGURE 6.7

Result returned from the temporary object.

A Card-Game Example

As a larger example of objects modeling the real world, let's look at a variation of the CARDS program from Chapter 4. This program, `CARDOBJ`, has been rewritten to use objects. It does not introduce any new concepts, but it does use almost all the programming ideas we've discussed up to this point.

As the CARDS example did, CARDOBJ creates three cards with fixed values and switches them around in an attempt to confuse the user about their location. But in CARDOBJ each card is an object of class card. Here's the listing:

```
// cardobj.cpp
// cards as objects #include
<iostream> using namespace std;
enum Suit { clubs, diamonds, hearts,
spades }; const int jack = 11;
//from 2 to 10 are const int queen =
12; //integers without names
const int king = 13; const int ace =
14;
//////////////////////////////// class card { private:
    int number; //2 to 10, jack, queen, king, ace Suit suit; //clubs, diamonds, hearts, spades public:
    card () //constructor (no args)
    { }
    //constructor (two args)
    card (int n, Suit s) : number(n), suit(s)
    { } void display(); //display card bool isEqual(card); //same as another card?
};
//-----void card::display() //display the card
{ if (number >= 2 && number <= 10 ) cout << number << " of ";
else switch(number) { case jack: cout << "jack of "; break; case queen: cout << "queen of
"; break; case king: cout << "king of "; break; case ace: cout << "ace of "; break;
} switch(suit) { case clubs: cout << "clubs"; break; case diamonds: cout <<
"diamonds"; break; case hearts: cout << "hearts"; break; case spades: cout <<
"spades"; break;
}
}
//-----bool card::isEqual(card c2) //return true if cards equal
{ return ( number==c2.number && suit==c2.suit ) ? true : false;
} //-----int main() { card temp, chosen, prize; //define
various cards int position;

card card1( 7, clubs ); //define & initialize card1 cout << "\nCard 1 is the "; card1.display();
//display card1

card card2( jack, hearts ); //define & initialize card2 cout << "\nCard 2 is the "; card2.display();
//display card2

card card3( ace, spades ); //define & initialize card3 cout << "\nCard 3 is the "; card3.display();
//display card3 prize = card3; //prize is the card to guess

cout << "\nI'm swapping card 1 and card 3"; temp = card3; card3 = card1; card1 = temp;

cout << "\nI'm swapping card 2 and card 3"; temp = card3; card3 = card2; card2 = temp;
```

```

cout << "\nI'm swapping card 1 and card 2"; temp = card2; card2 = card1; card1 = temp;

cout << "\nNow, where (1, 2, or 3) is the "; prize.display();           //display prize card
position;                       //get user's guess of position
cout << "? "; cin >>

switch (position)
{
    //set chosen to user's choice case 1: chosen = card1; break; case 2: chosen = card2; break;
    case 3: chosen = card3; break;
} if( chosen.isEqual(prize) ) //is chosen card the prize?
    cout << "That's right! You win!";
else cout << "Sorry. You lose.";
cout << " You chose the "; chosen.display();           //display chosen card
cout << endl; return 0;
}

```

There are two constructors in class `card`. The first, which takes no arguments, is used in `main()` to create the cards `temp`, `chosen`, and `prize`, which are not initialized. The second constructor, which takes two arguments, is used to create `card1`, `card2`, and `card3` and to initialize them to specific values. Besides the constructors, `card` has two other member functions, both defined outside the class.

The `display()` function takes no arguments; it simply displays the card object of which it is a member, using the number and suit data items in the card. The statement in `main()` `chosen.display();` displays the card chosen by the user.

The `isEqual()` function checks whether the card is equal to a card supplied as an argument. It uses the conditional operator to compare the card of which it is a member with a card supplied as an argument. This function could also have been written with an `if...else` statement

```

if( number==c2.number && suit==c2.suit ) return true;
else return false;

```

but the conditional operator is more compact.

In `isEqual()` the argument is called `c2` as a reminder that there are two cards in the comparison: The first card is the object of which `isEqual()` is a member. The expression `if( chosen.isEqual(prize) )` in `main()` compares the card `chosen` with the card `prize`.

Here's the output when the user guesses an incorrect card:

```

Card 1 is the 7 of clubs
Card 2 is the jack of hearts
Card 3 is the ace of spades
I'm swapping card 1 and card 3
I'm swapping card 2 and card 3
I'm swapping card 1 and card 2

```

Now, where (1, 2, or 3) is the ace of spades? 1
Sorry, you lose. You chose the 7 of clubs

Structures and Classes

The examples so far in this book have portrayed structures as a way to group data and classes as a way to group both data and functions. In fact, you can use structures in almost exactly the same way that you use classes. The only formal difference between `class` and `struct` is that in a class the members are private by default, while in a structure they are public by default.

Here's the format we've been using for classes:

```
class foo { private: int
    data1;
    public:
        void func();
};
```

Because `private` is the default in classes, this keyword is unnecessary. You can just as well write

```
class foo
{ int data1;
    public:
        void func();
};
```

and the `data1` will still be private. Many programmers prefer this style. We like to include the `private` keyword because it offers an increase in clarity.

If you want to use a structure to accomplish the same thing as this class, you can dispense with the keyword `public`, provided you put the public members before the private ones

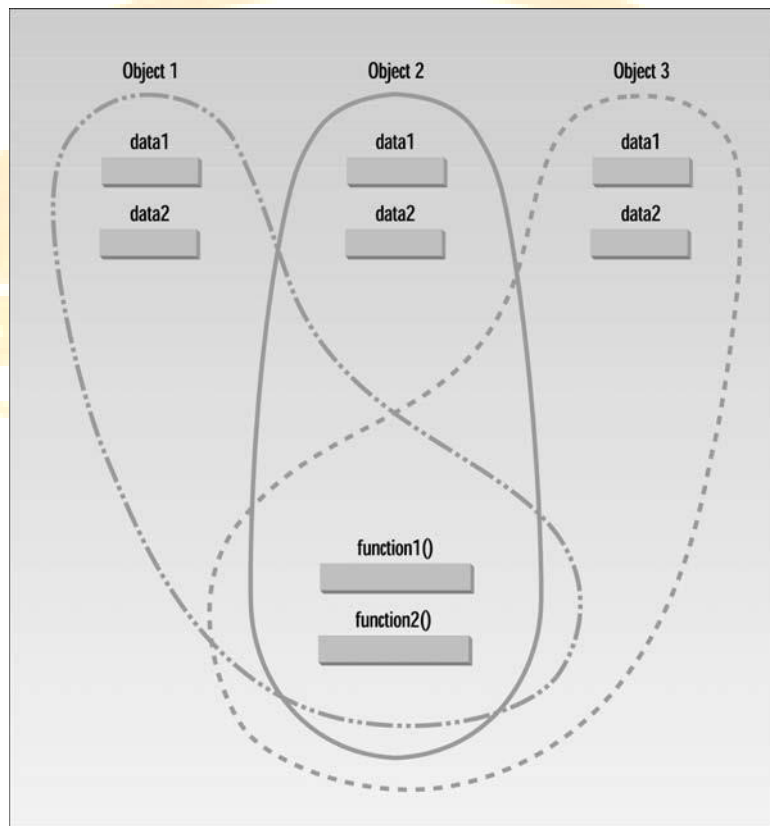
```
struct foo
{ void func();
    private:
        int data1;
};
```

since `public` is the default. However, in most situations programmers don't use a `struct` this way. They use structures to group only data, and classes to group both data and functions.

Classes, Objects, and Memory

We've probably given you the impression that each object created from a class contains separate copies of that class's data and member functions. This is a good first approximation, since it emphasizes that objects are complete, self-contained entities, designed using the class definition. The mental image here is of cars (objects) rolling off an assembly line, each one made according to a blueprint (the class definitions).

Actually, things are not quite so simple. It's true that each object has its own separate data items. On the other hand, contrary to what you may have been led to believe, all the objects in a given class use the same member functions. The member functions are created and placed in memory only once—when they are defined in the class definition. This makes sense; there's really no point in duplicating all the member functions in a class every time you create another object of that class, since the functions for each object are identical. The data items, however, will hold different values, so there must be a separate instance of each data item for each object. Data is therefore placed in memory when each object is defined, so there is a separate set of data for each object. Figure 6.8 shows how this looks.

**FIGURE 6.8**

Objects, data, functions, and memory.

In the `SMALLOBJ` example at the beginning of this chapter there are two objects of type `smallobj`, so there are two instances of `somedata` in memory. However, there is only one instance of the functions `setdata()` and `showdata()`. These functions are shared by all the objects of the class. There

is no conflict because (at least in a single-threaded system) only one function is executed at a time.

In most situations you don't need to know that there is only one member function for an entire class. It's simpler to visualize each object as containing both its own data and its own member functions. But in some situations, such as in estimating the size of an executing program, it's helpful to know what's happening behind the scenes.

Static Class Data

Having said that each object contains its own separate data, we must now amend that slightly. If a data item in a class is declared as `static`, only one such item is created for the entire class, no matter how many objects there are. A static data item is useful when all objects of the same class must share a common item of information. A member variable defined as `static` has characteristics similar to a normal static variable: It is visible only within the class, but its lifetime is the entire program. It continues to exist even if there are no objects of the class. (See Chapter 5 for a discussion of static variables.) However, while a normal static variable is used to retain information between calls to a function, static class member data is used to share information among the objects of a class.

Uses of Static Class Data

Why would you want to use static member data? As an example, suppose an object needed to know how many other objects of its class were in the program. In a road-racing game, for example, a race car might want to know how many other cars are still in the race. In this case a static variable `count` could be included as a member of the class. All the objects would have access to this variable. It would be the same variable for all of them; they would all see the same count.

An Example of Static Class Data

Here's an example, `STATDATA`, that demonstrates a simple static data member:

```
// statdata.cpp
// static class data #include
<iostream> using namespace std;
//////////////////////////////////// class foo
{
    private:
        static int count; //only one data item for all objects
                           //note: "declaration" only!

    public:
        foo()              //increments count when object created
```

```

        { count++; } int getcount()    //returns count
        { return count; }

};

//-----int foo::count = 0;    /*definition* of count
//////////////////////////////////// int main() { foo f1, f2, f3;    //create three objects

    cout << "count is " << f1.getcount() << endl; //each object cout << "count is " << f2.getcount()
    << endl; //sees the cout << "count is " << f3.getcount() << endl; //same value return 0;
}

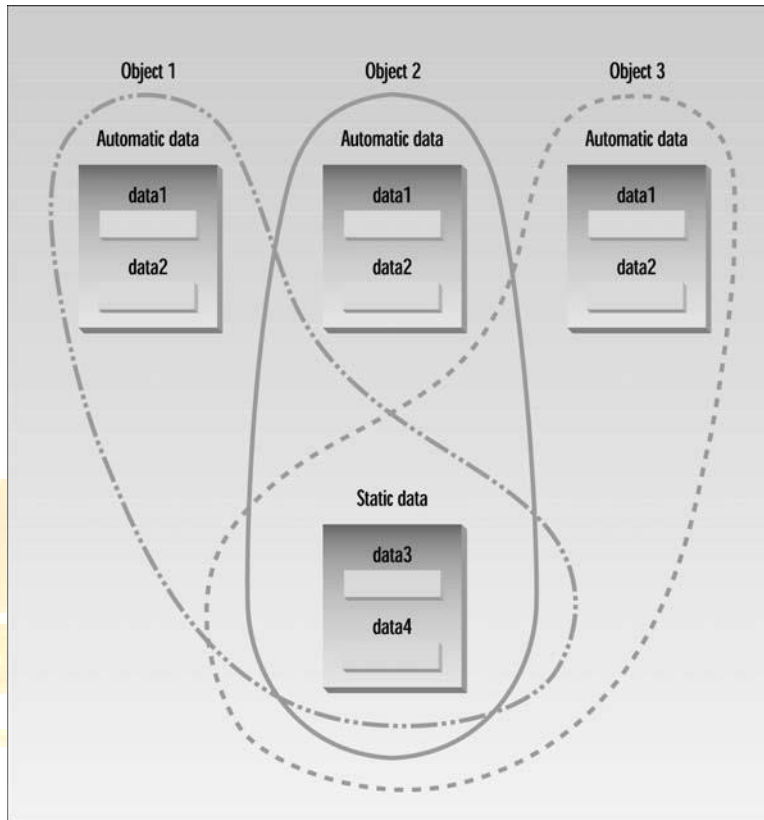
```

The class `foo` in this example has one data item, `count`, which is type `static int`. The constructor for this class causes `count` to be incremented. In `main()` we define three objects of class `foo`. Since the constructor is called three times, `count` is incremented three times. Another member function, `getcount()`, returns the value in `count`. We call this function from all three objects, and—as we expected—each prints the same value. Here’s the output: `count is 3` ← static data `count is 3` `count is 3`

If we had used an ordinary automatic variable—as opposed to a static variable—for `count`, each constructor would have incremented its own private copy of `count` once, and the output would have been

`count is 1` ← automatic data `count is 1` `count is 1`

Static class variables are not used as often as ordinary non-static variables, but they are important in many situations. Figure 6.9 shows how static variables compare with automatic variables.

**FIGURE 6.9**

Static versus automatic member variables.

Separate Declaration and Definition

Static member data requires an unusual format. Ordinary variables are usually declared (the compiler is told about their name and type) and defined (the compiler sets aside memory to hold the variable) in the same statement. Static member data, on the other hand, requires two separate statements. The variable's declaration appears in the class definition, but the variable is actually defined outside the class, in much the same way as a global variable.

Why is this two-part approach used? If static member data were defined inside the class (as it actually was in early versions of C++), it would violate the idea that a class definition is only a blueprint and does not set aside any memory. Putting the definition of static member data outside the class also serves to emphasize that the memory space for such data is allocated only

once, before the program starts to execute, and that one static member variable is accessed by an entire class; each object does not have its own version of the variable, as it would with ordinary member data. In this way a static member variable is more like a global variable.

It's easy to handle static data incorrectly, and the compiler is not helpful about such errors. If you include the declaration of a static variable but forget its definition, there will be no warning from the compiler. Everything looks fine until you get to the linker, which will tell you that you're trying to reference an undeclared global variable. This happens even if you include the definition but forget the class name (the `foo::` in the `STATDATA` example).

const and Classes

We've seen several examples of `const` used on normal variables to prevent them from being modified, and in Chapter 5 we saw that `const` can be used with function arguments to keep a function from modifying a variable passed to it by reference. Now that we know about classes, we can introduce some other uses of `const`: on member functions, on member function arguments, and on objects. These concepts work together to provide some surprising benefits. **const**

Member Functions

A `const` member function guarantees that it will never modify any of its class's member data. The `CONSTFU` program shows how this works.

```
//constfu.cpp
//demonstrates const member functions
/ class aClass { private:
int alpha;
public:
    void nonFunc()    //non-const member function
        { alpha = 99; } //OK

    void conFunc() const //const member function
        { alpha = 99; } //ERROR: can't modify a member };
```

The non-const function `nonFunc()` can modify member data `alpha`, but the constant function `conFunc()` can't. If it tries to, a compiler error results.

A function is made into a constant function by placing the keyword `const` after the declarator but before the function body. If there is a separate function declaration, `const` must be used in both declaration and definition. Member functions that do nothing but acquire data from an object are obvious candidates for being made `const`, because they don't need to modify any data.

Making a function `const` helps the compiler flag errors, and tells anyone looking at the listing that you intended the function not to modify anything in its object. It also makes possible the creation and use of `const` objects, which we'll discuss soon.

A Distance Example

To avoid raising too many subjects at once we have, up to now, avoided using `const` member functions in the example programs. However, there are many places where `const` member functions should be used. For example, in the `Distance` class, shown in several programs, the `showdist()` member function could be made `const` because it doesn't (or certainly shouldn't!) modify any of the data in the object for which it was called. It should simply display the data.

Also, in `ENGLRET`, the `add_dist()` function should not modify any of the data in the object for which it was called. This object should simply be added to the object passed as an argument, and the resulting sum should be returned. We've modified the `ENGLRET` program to show how these two constant functions look. Note that `const` is used in both the declaration and the definition of `add_dist()`. Here's the listing for `ENGCONST`:

```
// engConst.cpp
// const member functions and const arguments to member functions
#include <iostream> using
namespace std;
//////////////////// class Distance                //English Distance class
{ private: int feet; float
  inches;
public:
    //constructor (no args)
    Distance() : feet(0), inches(0.0)
    { }
    //constructor (two args)
    Distance(int ft, float in) : feet(ft), inches(in)
    { }

    void getdist()           //get length from user
    { cout << "\nEnter feet: "; cin >> feet; cout << "Enter inches:
      "; cin >> inches;
    } void showdist() const   //display distance
    { cout << feet << "\n" << inches << "\n"; }

    Distance add_dist(const Distance&) const; //add };
//-----
//add this distance to d2, return the sum
Distance Distance::add_dist(const Distance& d2) const
{
    Distance temp;           //temporary variable

    // feet = 0;              //ERROR: can't modify this
    // d2.feet = 0;           //ERROR: can't modify d2
    inches = inches + d2.inches; //add the
    inches if(temp.inches >= 12.0) //if total exceeds 12.0,
    {                               //then decrease inches temp.inches -= 12.0;    //by 12.0
    and temp.feet = 1;              //increase feet }                      //by 1 temp.feet +=
    feet + d2.feet; //add the feet return temp;
}
```

```

//////////////////////////////////// int main()
{
    Distance dist1, dist3;    //define two lengths Distance dist2(11, 6.25);    //define,
    initialize dist2

    dist1.getdist();          //get dist1 from user dist3 = dist1.add_dist(dist2); //dist3 =
    dist1 + dist2

                                //display all lengths
    cout << "\ndist1 = "; dist1.showdist(); cout << "\ndist2 = ";
    dist2.showdist(); cout << "\ndist3 = "; dist3.showdist(); cout
    << endl; return 0;
}

```

Here, `showdist()` and `add_dist()` are both constant member functions. In `add_dist()` we show in the first commented statement, `feet = 0`, that a compiler error is generated if you attempt to modify any of the data in the object for which this constant function was called.

const Member Function Arguments

We mentioned in Chapter 5 that if an argument is passed to an ordinary function by reference, and you don't want the function to modify it, the argument should be made `const` in the function declaration (and definition). This is true of member functions as well. In `ENGCONST` the argument to `add_dist()` is passed by reference, and we want to make sure that `ENGCONST` won't modify this variable, which is `dist2` in `main()`. Therefore we make the argument `d2` to `add_dist()` `const` in both declaration and definition. The second commented statement shows that the compiler will flag as an error any attempt by `add_dist()` to modify any member data of its argument `dist2`.

const Objects

In several example programs, we've seen that we can apply `const` to variables of basic types such as `int` to keep them from being modified. In a similar way, we can apply `const` to objects of classes. When an object is declared as `const`, you can't modify it. It follows that you can use only `const` member functions with it, because they're the only ones that guarantee not to modify it. The `CONSTOBJ` program shows an example.

```

// constObj.cpp
// constant Distance objects #include
<iostream> using namespace std;
//////////////////////////////////// class Distance    //English Distance class
{ private: int feet; float inches; public:            //2-arg constructor
    Distance(int ft, float in) : feet(ft), inches(in)
    { } void getdist()    //user input; non-const func
    { cout << "\nEnter feet: "; cin >> feet; cout << "Enter
    inches: "; cin >> inches;
    } void showdist() const    //display distance; const func
    { cout << feet << "\." << inches << "\n"; } };

```

```
//////////////////////////////// int main() { const Distance football(300, 0);

// football.getdist();      //ERROR: getdist() not const cout << "football = ";
    football.showdist();    //OK cout << endl; return 0;
}
```

A football field (for American-style football) is exactly 300 feet long. If we were to use the length of a football field in a program, it would make sense to make it `const`, because changing it would represent the end of the world for football fans. The `CONSTOBJ` program makes `football` a `const` variable. Now only `const` functions, such as `showdist()`, can be called for this object. Non-`const` functions, such as `getdist()`, which gives the object a new value obtained from the user, are illegal. In this way the compiler enforces the `const` value of `football`.

When you're designing classes it's a good idea to make `const` any function that does not modify any of the data in its object. This allows the user of the class to create `const` objects. These objects can use any `const` function, but cannot use any non-`const` function. Remember, using `const` helps the compiler to help you.

What Does It All Mean?

Now that you've been introduced to classes and objects, you may wonder what benefit they really offer. After all, as you can see by comparing several of the programs in this chapter with those in Chapter 4, it's possible to do the same sorts of things with a procedural approach as it is with objects.

One benefit of OOP that you may have glimpsed already is the close correspondence between the real-world things being modeled by the program and the C++ objects *in* the program. A widget part object in a program represents a widget part in the real world, a card object represents a card, a circle object represents a graphics circle, and so on. In C++, everything about a widget part is included in its class description—the part number and other data items, and the functions necessary to access and operate on this data. This makes it easy to conceptualize a programming problem. You figure out what parts of the problem can be most usefully represented as objects, and then put all the data and functions connected with that object into the class. If you're using a C++ class to represent a playing card, you put into this class the data items that represent the value of the card, and also the functions to set value, retrieve it, display it, compare it, and so on.

In a procedural program, by contrast, the global variables and functions connected with a realworld object are distributed all over the listing; they don't form a single, easily grasped unit.

In some situations it may not be obvious what parts of a real-life situation should be made into objects. If you're writing a program that plays chess, for instance, what are the objects? The chessmen, the squares on the board, or possibly entire board positions?

In small programs, such as many of the ones in this book, you can often proceed by trial and error. You break a problem into objects in one way and write trial class definitions for these objects. If the classes seem to match reality in a useful way, you continue. If they don't, you may need to start over, selecting different entities to be classes. The more experience you have with OOP, the easier it will be to break a programming problem into classes.

Larger programs may prove too complex for this trial-and-error approach. A new field, object-oriented design (OOD) is increasingly applied to analyzing a programming problem and figuring out what classes and objects should be used to represent the real-world situation (which is often called the *problem domain*). We'll discuss this methodology in detail in Chapter 16, "Object-Oriented Software Development."

Some of the benefits of object-oriented programming are probably not apparent at this point. Remember that OOP was devised to cope with the complexity of large programs. Smaller programs, such as the examples in this chapter, have less need for the organizational power that OOP provides. The larger the program, the greater the benefit. But even for small programs, once you start thinking in object-oriented terms, the OO design approach becomes natural and surprisingly helpful. One advantage is that in an OO program the compiler can find many more conceptual errors than in a procedural program.

Summary

A class is a specification or blueprint for a number of objects. Objects consist of both data and functions that operate on that data. In a class definition, the members—whether data or functions—can be *private*, meaning they can be accessed only by member functions of that class, or *public*, meaning they can be accessed by any function in the program.

A member function is a function that is a *member* of a class. Member functions have access to an object's private data, while non-member functions do not.

A constructor is a member function, with the same name as its class, that is executed every time an object of the class is created. A constructor has no return type but can take arguments. It is often used to give initial values to object data members. Constructors can be overloaded, so an object can be initialized in different ways.

A destructor is a member function with the same name as its class but preceded by a tilde (~). It is called when an object is destroyed. A destructor takes no arguments and has no return value.

In the computer's memory there is a separate copy of the data members for each object that is created from a class, but there is only one copy of a class's member functions. You can restrict a data item to a single instance for all objects of a class by making it *static*.

One reason to use OOP is the close correspondence between real-world objects and OOP classes. Deciding what objects and classes to use in a program can be complicated. For small programs, trial and error may be sufficient. For large programs, a more systematic approach is usually needed.

Questions

Answers to these questions can be found in Appendix G.

1. What is the purpose of a class definition?
2. A _____ has the same relation to an _____ that a basic data type has to a variable of that type.
3. In a class definition, data or functions designated *private* are accessible
 - a. to any function in the program.
 - b. only if you know the password.
 - c. to member functions of that class.
 - d. only to public members of the class.
4. Write a class definition that creates a class called *leverage* with one private data member, *crowbar*, of type *int* and one public function whose declaration is *void pry()*.
5. True or false: Data items in a class must be private.
6. Write a statement that defines an object called *lever1* of the *leverage* class described in Question 4.
7. The dot operator (or class member access operator) connects the following two entities (reading from left to right):
 - a. A class member and a class object
 - b. A class object and a class
 - c. A class and a member of that class
 - d. A class object and a member of that class
8. Write a statement that executes the *pry()* function in the *lever1* object, as described in Questions 4 and 6.
9. Member functions defined inside a class definition are _____ by default.

10. Write a member function called `getcrow()` for the `leverage` class described in Question 4. This function should return the value of the `crowbar` data. Assume the function is defined within the class definition.
11. A constructor is executed automatically when an object is _____.
12. A constructor's name is the same as _____.
13. Write a constructor that initializes to 0 the `crowbar` data, a member of the `leverage` class described in Question 4. Assume that the constructor is defined within the class definition.
14. True or false: In a class you can have more than one constructor with the same name.
15. A member function can always access the data
 - a. in the object of which it is a member.
 - b. in the class of which it is a member.
 - c. in any object of the class of which it is a member.
 - d. in the public part of its class.
16. Assume that the member function `getcrow()` described in Question 10 is defined outside the class definition. Write the declaration that goes inside the class definition.
17. Write a revised version of the `getcrow()` member function from Question 10 that is defined outside the class definition.
18. The only technical difference between structures and classes in C++ is that _____.
19. If three objects of a class are defined, how many copies of that class's data items are stored in memory? How many copies of its member functions?
20. Sending a message to an object is the same as _____.
21. Classes are useful because they
 - a. are removed from memory when not in use.
 - b. permit data to be hidden from other classes.
 - c. bring together all aspects of an entity in one place.
 - d. can closely model objects in the real world.
22. True or false: There is a simple but precise methodology for dividing a real-world programming problem into classes.
23. For the object for which it was called, a `const` member function
 - a. can modify both `const` and non-`const` member data.
 - b. can modify only `const` member data.
 - c. can modify only non-`const` member data.

- d. can modify neither `const` nor `non-const` member data.
24. True or false: If you declare a `const` object, it can only be used with `const` member functions.
25. Write a declaration (not a definition) for a `constvoid` function called `aFunc()` that takes one `const` argument called `jerry` of type `float`.

Exercises

Answers to the starred exercises can be found in Appendix G.

- *1. Create a class that imitates part of the functionality of the basic data type `int`. Call the class `Int` (note different capitalization). The only data in this class is an `int` variable. Include member functions to initialize an `Int` to 0, to initialize it to an `int` value, to display it (it looks just like an `int`), and to add two `Int` values.

Write a program that exercises this class by creating one uninitialized and two initialized `Int` values, adding the two initialized values and placing the response in the uninitialized value, and then displaying this result.

- *2. Imagine a tollbooth at a bridge. Cars passing by the booth are expected to pay a 50 cent toll. Mostly they do, but sometimes a car goes by without paying. The tollbooth keeps track of the number of cars that have gone by, and of the total amount of money collected.

Model this tollbooth with a class called `tollBooth`. The two data items are a type `unsigned int` to hold the total number of cars, and a type `double` to hold the total amount of money collected. A constructor initializes both of these to 0. A member function called `payingCar()` increments the car total and adds 0.50 to the cash total. Another function, called `noPayCar()`, increments the car total but adds nothing to the cash total. Finally, a member function called `display()` displays the two totals. Make appropriate member functions `const`.

Include a program to test this class. This program should allow the user to push one key to count a paying car, and another to count a nonpaying car. Pushing the Esc key should cause the program to print out the total cars and total cash and then exit.

- *3. Create a class called `time` that has separate `int` member data for hours, minutes, and seconds. One constructor should initialize this data to 0, and another should initialize it to fixed values. Another member function should display it, in 11:59:59 format. The final member function should add two objects of type `time` passed as arguments.

A `main()` program should create two initialized `time` objects (should they be `const`?) and one that isn't initialized. Then it should add the two initialized values together, leaving the result in the third `time` variable. Finally it should display the value of this third variable. Make appropriate member functions `const`.

4. Create an employee class, basing it on Exercise 4 of Chapter 4. The member data should comprise an `int` for storing the employee number and a `float` for storing the employee's compensation. Member functions should allow the user to enter this data and display it. Write a `main()` that allows the user to enter data for three employees and display it.
5. Start with the date structure in Exercise 5 in Chapter 4 and transform it into a date class. Its member data should consist of three `ints`: month, day, and year. It should also have two member functions: `getdate()`, which allows the user to enter a date in 12/31/02 format, and `showdate()`, which displays the date.
6. Extend the employee class of Exercise 4 to include a date class (see Exercise 5) and an `etype` enum (see Exercise 6 in Chapter 4). An object of the date class should be used to hold the date of first employment; that is, the date when the employee was hired. The `etype` variable should hold the employee's type: laborer, secretary, manager, and so on. These two items will be private member data in the employee definition, just like the employee number and salary. You'll need to extend the `getemploy()` and `putemploy()` functions to obtain this new information from the user and display it. These functions will probably need `switch` statements to handle the `etype` variable. Write a `main()` program that allows the user to enter data for three employee variables and then displays this data.
7. In ocean navigation, locations are measured in degrees and minutes of latitude and longitude. Thus if you're lying off the mouth of Papeete Harbor in Tahiti, your location is 149 degrees 34.8 minutes west longitude, and 17 degrees 31.5 minutes south latitude. This is written as 149°34.8' W, 17°31.5' S. There are 60 minutes in a degree. (An older system also divided a minute into 60 seconds, but the modern approach is to use decimal minutes instead.) Longitude is measured from 0 to 180 degrees, east or west from Greenwich, England, to the international dateline in the Pacific. Latitude is measured from 0 to 90 degrees, north or south from the equator to the poles.

Create a class `angle` that includes three member variables: an `int` for degrees, a `float` for minutes, and a `char` for the direction letter (N, S, E, or W). This class can hold either a latitude variable or a longitude variable. Write one member function to obtain an angle value (in degrees and minutes) and a direction from the user, and a second to display the angle value in 179°59.9' E format. Also write a three-argument constructor. Write a `main()` program that displays an angle initialized with the constructor, and then, within a loop, allows the user to input any angle value, and then displays the value. You can use the hex character constant `'\xF8'`, which usually prints a degree (°) symbol.

8. Create a class that includes a data member that holds a "serial number" for each object created from the class. That is, the first object created will be numbered 1, the second 2, and so on.

To do this, you'll need another data member that records a count of how many objects have been created so far. (This member should apply to the class as a whole; not to individual objects. What keyword specifies this?) Then, as each object is created, its constructor can examine this count member variable to determine the appropriate serial number for the new object.

Add a member function that permits an object to report its own serial number. Then write a `main()` program that creates three objects and queries each one about its serial number. They should respond I am object number 2, and so on.

9. Transform the `fraction` structure from Exercise 8 in Chapter 4 into a `fraction` class. Member data is the `fraction`'s numerator and denominator. Member functions should accept input from the user in the form $3/5$, and output the `fraction`'s value in the same format. Another member function should add two `fraction` values. Write a `main()` program that allows the user to repeatedly input two fractions and then displays their sum. After each operation, ask whether the user wants to continue.
10. Create a class called `ship` that incorporates a ship's number and location. Use the approach of Exercise 8 to number each `ship` object as it is created. Use two variables of the `angle` class from Exercise 7 to represent the ship's latitude and longitude. A member function of the `ship` class should get a position from the user and store it in the object; another should report the serial number and position. Write a `main()` program that creates three ships, asks the user to input the position of each, and then displays each ship's number and position.
11. Modify the four-function `fraction` calculator of Exercise 12 in Chapter 5 to use a `fraction` class rather than a structure. There should be member functions for input and output, as well as for the four arithmetical operations. While you're at it, you might as well install the capability to reduce fractions to lowest terms. Here's a member function that will reduce the `fraction` object of which it is a member to lowest terms. It finds the greatest common divisor (gcd) of the `fraction`'s numerator and denominator, and uses this gcd to divide both numbers.

```
void fraction::lowterms() // change ourself to lowest terms
{ long tnum, tden, temp, gcd;

    tnum = labs(num); // use non-negative copies
    tden = labs(den); // (needs
    cmath) if(tden==0) // check for n/0
        { cout << "Illegal fraction: division by 0"; exit(1); }
    else if( tnum==0 ) // check
        for 0/n
        { num=0; den = 1; return; }

    // this 'while' loop finds the gcd of tnum and tden while(tnum != 0)
    { if(tnum < tden) // ensure numerator larger
        { temp=tnum; tnum=tden; tden=temp; } // swap them
        tnum = tnum - tden; // subtract them
```

```

    } gcd = tden;           // this is greatest common divisor num = num / gcd;      //
    divide both num and den by gcd den = den / gcd;      // to reduce frac to lowest terms }

```

You can call this function at the end of each arithmetic function, or just before you perform output. You'll also need the usual member functions: four arithmetic operations, input, and display. You may find a two-argument constructor useful.

12. Note that one advantage of the OOP approach is that an entire class can be used, without modification, in a different program. Use the `fraction` class from Exercise 11 in a program that generates a multiplication table for fractions. Let the user input a denominator, and then generate all combinations of two such fractions that are between 0 and 1, and multiply them together. Here's an example of the output if the denominator is 6:

	1/6	1/3	1/2	2/3	5/6
1/6	1/36	1/18	1/12	1/9	5/36
1/3	1/18	1/9	1/6	2/9	5/18
1/2	1/12	1/6	1/4	1/3	5/12
2/3	1/9	2/9	1/3	4/9	5/9
5/6	5/36	5/18	5/12	5/9	25/36